

Software Requirements Specification

for Smart Pest Management System

Version: 1.0

Prepared by: Antigravity & User

Date: 2026-01-30

1. Introduction

1.1 Purpose

The purpose of this document is to define the software requirements for the **Smart Pest Management System**. This web application is designed to assist farmers in identifying crop diseases using AI, providing treatment recommendations, managing pesticide shops, and facilitating expert consultations.

1.2 Document Conventions

This document uses the following conventions: - **Bold** text indicates important terms or headings. - *Italic* text indicates emphasis. - The term "System" refers to the Smart Pest Management System.

1.3 Intended Audience and Reading Suggestions

- **Developers:** To understand the functional and non-functional requirements for implementation.
- **Project Managers:** To track progress and ensure scope is met.
- **Stakeholders/Farmers:** To understand the features provided by the system.
- **Testers:** To verify the system against the requirements.

1.4 Product Scope

The system is a web-based platform with three primary interfaces: 1. **Farmer Interface:** Disease detection, chatbot assistance, forum access, and shop locator. 2. **Expert Interface:** Managing consultations and replying to farmer queries. 3. **Admin Interface:** User management, database updates (shops, medicines, subsidies), and system monitoring.

1.5 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
 - Project Source Code and Documentation.
-

2. Overall Description

2.1 Product Perspective

This system is a standalone web application. It operates locally (offline-capable for core features) but is designed to be deployment-ready. It integrates a custom-trained Convolutional Neural Network (CNN) for offline disease detection and a local rule-based chatbot.

2.2 Product Functions

- **Disease Detection:** Analyze leaf images to identify diseases.
- **Treatment Recommendations:** Provide organic and inorganic solutions.
- **Chatbot Assistance:** Answer queries offline using fuzzy logic.
- **Expert Consultation:** Connect farmers with agricultural experts.
- **Information Portal:** Display weather, shops, and government subsidies.
- **Administration:** Manage users, content, and system resources.

2.3 User Classes and Characteristics

- **Farmer:** Primary user. Needs simple, visual interfaces. Uses the app for diagnosis and advice.
- **Agricultural Expert:** Specialized user. Reviews consultation requests and provides expert advice.
- **Administrator:** Technical user. Manages user accounts, ensures data integrity, and updates static content.

2.4 Operating Environment

- **Client:** Modern Web Browser (Chrome, Firefox, Edge).
- **Server:** Python 3.9+ runtime, MongoDB instance.
- **Hardware:** Capable of running a local python server (for local deployment) or standard web server specs.

2.5 Design and Implementation Constraints

- **Offline Capability:** Core features (AI, Chatbot) must work without internet.
- **Database:** Uses MongoDB; requires a local instance.
- **Performance:** AI Inference must be optimized for local CPU execution.

2.6 User Documentation

- In-app help and walkthroughs are provided via the Chatbot and UI hints.

2.7 Assumptions and Dependencies

- Users have a device with a camera and web browser.
- Local Python environment is correctly set up with dependencies.
- MongoDB service is running in the background.

3. External Interface Requirements

3.1 User Interfaces

- **Design:** Clean, nature-themed UI using Green/White color palette.
- **Responsive:** Mobile-friendly layout for field use.
- **Components:** Floating chatbot, Image upload drag-and-drop, Dashboard tables.

3.2 Hardware Interfaces

- **Camera:** Browser access to device camera for capturing leaf images.
- **Storage:** Local disk access for storing uploaded images and logs.

3.3 Software Interfaces

- **Database:** MongoDB connection via pymongo.
- **AI Libraries:** PyTorch (torch, torchvision) for model inference.
- **Web Framework:** Flask (Backend) and React (Frontend).

3.4 Communications Interfaces

- **HTTP/REST:** Communication between React Frontend and Flask Backend via `localhost:5000`.
- **JSON:** Data exchange format.

4. System Features

4.1 Disease Detection (AI)

4.1.1 Description and Priority

High Priority. The core feature allowing farmers to detect crop diseases from photos.

4.1.2 Stimulus/Response Sequences

- **Stimulus:** User uploads an image.
- **Response:** System preprocesses image, runs inference, and returns disease class + confidence.

4.1.3 Functional Requirements

- REQ-1: System shall accept .jpg and .png image formats.
- REQ-2: System shall use the loaded MobileNetV2 model for prediction.
- REQ-3: System shall provide a confidence score for the prediction.

4.2 Local Chatbot

4.2.1 Description and Priority

High Priority. Provides instant assistance without internet.

4.2.2 Stimulus/Response Sequences

- **Stimulus:** User types a question (e.g., "Tomato treatment").
- **Response:** System matches keywords using `diffLib` and returns relevant advice.

4.2.3 Functional Requirements

- REQ-4: Chatbot shall operate without external API keys.
- REQ-5: Chatbot shall interpret queries related to crops, diseases, shops, and subsidies.

4.3 Expert Consultation

4.3.1 Description and Priority

Medium Priority. Connects users to human experts for complex cases.

4.3.2 Stimulus/Response Sequences

- **Stimulus:** Farmer submits a consultation request.
- **Response:** Request is stored in DB. Expert views it and replies.

4.3.3 Functional Requirements

- REQ-6: Users must be logged in to request consultation.
- REQ-7: Experts shall have a dedicated dashboard to view pending requests.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- **Response Time:** AI Prediction should take < 2 seconds on standard hardware.
- **Throughput:** Backend should handle multiple concurrent requests without crashing.

5.2 Safety Requirements

- **Validation:** All user inputs (files, text) must be sanitized to prevent injection attacks.
- **Error Handling:** System must fail gracefully and provide readable error messages.

5.3 Security Requirements

- **Data Privacy:** User passwords must be hashed (if enabled in production).
- **Access Control:** Admin routes must be protected from unauthorized access.

5.4 Software Quality Attributes

- **Usability:** Interface must be intuitive for non-technical users.
- **Maintainability:** Backend code is modular (routes separated from logic).
- **Reliability:** Core features must function independently of external services (Offline First).

Appendix A: Glossary

- **SRS:** Software Requirements Specification

- **AI:** Artificial Intelligence
- **CNN:** Convolutional Neural Network
- **UI:** User Interface